Drishyamitra AI-Powered Photo Management System

Project Description:

Drishyamitra is a smart, AI-powered photo management and retrieval platform designed to automate how users organize, search, and interact with their digital memory libraries. Built on a modern full-stack architecture, it pairs a responsive React.js frontend styled with custom Vanilla CSS and HSL design tokens with a robust Python Flask backend. By replacing traditional manual albums with intelligent automation, the system provides a seamless, dynamic, and highly interactive gallery experience.

At its core, the platform integrates advanced computer vision pipelines using DeepFace (RetinaFace detector and Facenet512 model) alongside OpenCV to detect, crop, and identify individuals. Unidentified background faces are grouped automatically using DBSCAN clustering, enabling users to easily label or merge collections. Additionally, OpenAI's GPT-4o-mini Vision API processes newly uploaded photos to extract context, auto-generate descriptions, and suggest logical tags and event categories.

To make retrieval fast and intuitive, Drishyamitra implements a hybrid search engine combining ChromaDB vector storage for semantic embeddings and SQLite with SQLAlchemy for structured metadata search. Users can query their library conversationally or utilize a multilingual Voice Search Engine (Web Speech API) supporting English, Hindi, Spanish, and Telugu. Interaction is further enhanced by an agentic conversational AI assistant built with Groq (Llama 3.3), LangChain, and LangGraph, where specialized agents collaborate to analyze images, query database stats, and perform actions.

Beyond core management, the application features scene and color-histogram-based event categorization, a live storage breakdown dashboard, and real-time activity analytics. It supports secure JWT authentication, Cloudinary image hosting, and one-click photo sharing via Gmail (SMTP) and WhatsApp (Twilio API). A built-in ZIP backup utility ensures users can securely export their entire digital library, making Drishyamitra a complete, secure, and modern photo companion.

.

Scenario 1: College Event Photo Management and Smart Retrieval

During his college's annual tech fest, event coordinator Rahul uses Drishyamitra to efficiently manage, categorize, and organize thousands of photos captured across competitions and workshops. Upon uploading the massive image library, the platform's computer vision pipeline automatically recognizes and groups identical faces, while visual similarity clustering groups photos into thematic albums like "Technical Events" or "Prize Distribution." Semantic indexing and vector embeddings are generated on the fly, transforming a chaotic collection of files into a fully structured, searchable database without manual intervention.

To handle student and faculty requests, Rahul relies on Drishyamitra's conversational AI assistant to perform natural language searches such as "Show photos of students receiving prizes during the coding competition," which instantly retrieves the correct images. The system's analytics dashboard provides real-time event statistics, and Rahul can easily share individual photos or curated albums via email or WhatsApp directly through the chatbot. Ultimately, Drishyamitra turns a tedious multi-day sorting task into a fast, intuitive experience, streamlining large-scale event photography management and sharing.

Scenario 2: Multi-Generational Family Archival & Multilingual Voice Search

Priya uses Drishyamitra to archive decades of family photos and scanned prints into a digital museum. The system automatically detects and groups faces over time, allowing her to merge childhood portraits with modern images using DBSCAN clustering. Simultaneously, GPT-4o-mini Vision auto-tags landmarks and organizes photos into event categories like weddings and vacations, creating a structured heritage vault.

To ensure accessibility, Priya's family members use the multilingual Voice Search Engine to find memories by speaking commands in Hindi or Telugu. The conversational AI assistant retrieves specific photos instantly, bypassing manual browsing. Priya also monitors cloud storage via the analytics dashboard and keeps the archive secure with the ZIP backup utility, bridging generational and linguistic gaps for a shared family legacy.

Scenario 3: Family Memory Archiving & Multilingual Voice Accessibility (Personal Use Case)

Devi, a grandmother, wants to find photos of her grandchildren from a family wedding celebrated years ago. Because she struggles with typing on digital devices, she uses Drishyamitra's voice search feature. By tapping the microphone icon, the browser clears mic permissions and utilizes the Web Speech API to capture her voice as she speaks in Hindi: *"Mere bacchon ki shaadi ki photos dikhao."* The system automatically strips any trailing punctuation from the transcription and forwards the clean query to the AI assistant backend.

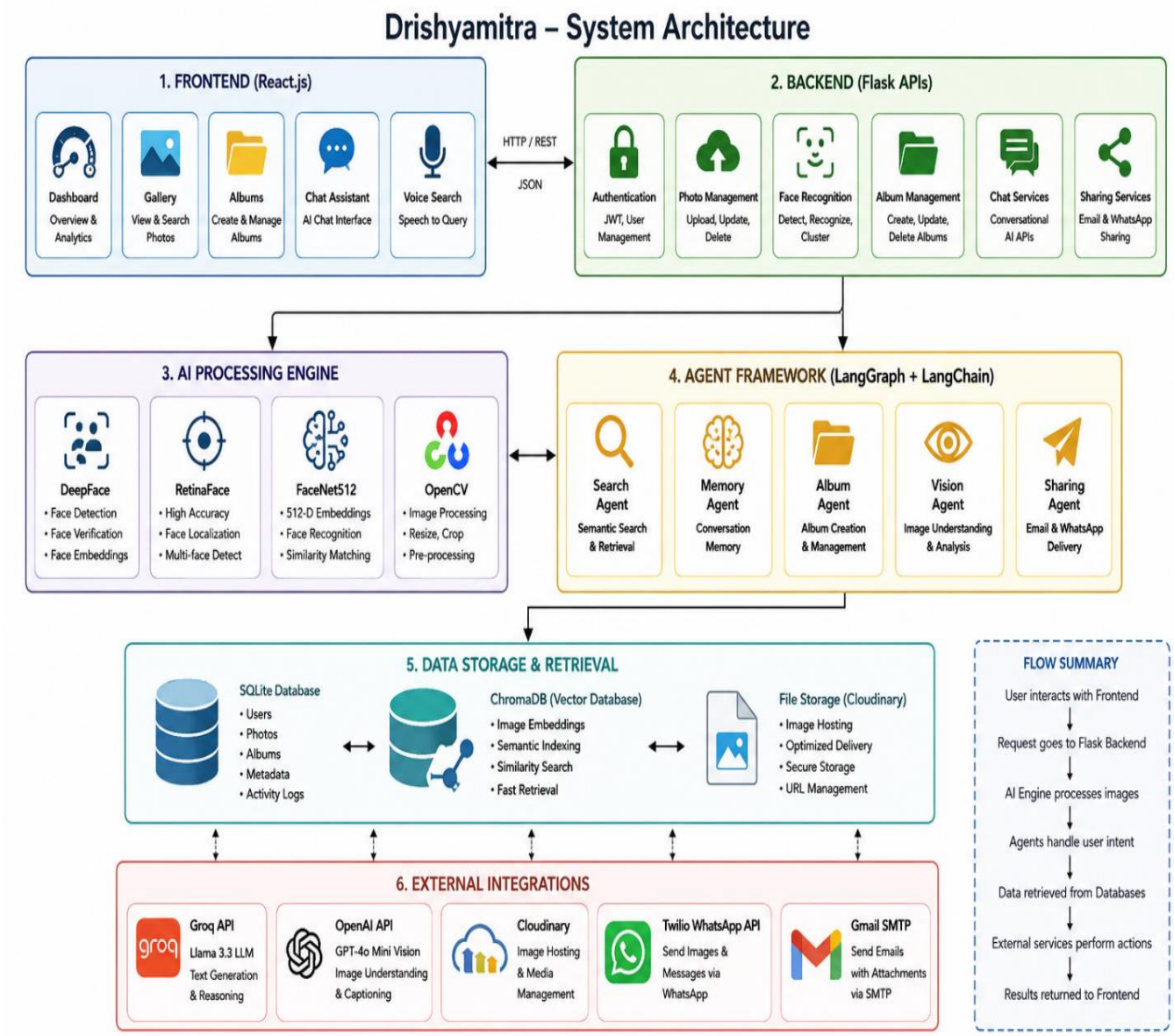
The LangGraph-powered conversational assistant processes the query, identifying the dual intent of locating grandchildren face profiles and filtering by the "Weddings" album. Instead of filling the chat history with duplicate image attachments, the search agent communicates directly with the gallery interface to trigger a viewport redirection. Within seconds, Devi's tablet screen dynamically refreshes to display only the relevant wedding photographs of her grandchildren, providing an accessible and hands-free photo retrieval experience.

Scenario 4: Corporate Media Management & Collaboration

Amit, a freelance photographer, uploads 200 photos from an outdoor baby shower to Drishyamitra to simplify post-shoot delivery. Upon upload, the system automatically analyzes the image backgrounds—such as garden landscapes and balloons—using computer vision and color histograms to group the photos into a scene-based "Events" album. Simultaneously, DeepFace and DBSCAN clustering group all occurrences of the mother's face into a single profile, which Amit quickly labels with her name.

To deliver the files, Amit opens the AI Chat and speaks a natural language query to filter the gallery. The system's Search Agent retrieves the relevant images and instantly updates his screen to show only the photos featuring the mother. Amit then instructs the chatbot to share the photos with the client's email address. The Sharing Agent automatically detects the active photos on his screen, packages them, and delivers high-resolution copies via SMTP email in a single step, saving hours of manual sorting and archiving.

Architecture :



Pre-requisites:

1. **Python Environment Setup**: Install **Python 3.10 or higher**. Create and activate a Python virtual environment (**.venv**) inside the drishyamitra-backend directory to isolate dependencies.

2. **Node.js Installation**: Install **Node.js 18 or higher** (which includes **npm 9+**) on your machine to handle frontend compilation.

3. **Backend Dependencies**: Navigate to the drishyamitra-backend directory and run the command **pip install -r requirements.txt** to install backend packages including **Flask**, **DeepFace**, **ChromaDB**, **LangGraph**, and **Groq**.

4. **Frontend Dependencies**: Navigate to the drishyamitra-frontend directory and run the command **npm install** to download and install the required React packages.

5. **Database Setup**: The system is pre-configured to automatically initialize a SQLite database named **drishyamitra.db** in the backend root directory upon startup. If you wish to use PostgreSQL, override this default by specifying your connection string in the **DATABASE_URL** environment variable.

6. **Backend Configuration**: Create a .env file in the backend directory and populate it with credentials such as **GROQ_API_KEY**, **OPENAI_API_KEY**, **CLOUDINARY_URL**, **SMTP_USER**, and **TWILIO_AUTH_TOKEN**.

7. **Frontend Configuration**: Create a .env file in the frontend directory and add the variable **REACT_APP_API_BASE_URL=http://localhost:5000** to direct the Axios client to the Flask API.

8. **Backend Launch**: Run the Flask server by activating your virtual environment in the backend folder and executing the command **python app.py**.

9. **Frontend Launch**: Start the React development server by opening a separate terminal, navigating to the frontend folder, and executing the command **npm start**.

10. **Verification**: Verify the setup by accessing the backend API status endpoint at http://localhost:5000 and the frontend application dashboard at http://localhost:3000.

Project WorkFlow

Milestone 1: Environment Setup and Project Initialization

- **Activity 1.1**: Create and activate a Python virtual environment using python -m venv .venv inside the drishyamitra-backend directory.
- **Activity 1.2**: Initialize the project folder structure with directories for backend agents, workflows, routes, database models, services, and frontend source components and styling stylesheets.
- **Activity 1.3**: Configure the backend .env file with required credentials (GROQ_API_KEY, OPENAI_API_KEY, CLOUDINARY_URL, SMTP user/password, and Twilio tokens) and the frontend .env file with REACT_APP_API_BASE_URL=http://localhost:5000.
- **Activity 1.4**: Install project dependencies by running pip install -r requirements.txt in the backend directory and npm install in the frontend directory.
- **Activity 1.5**: Verify environment setup and ensure database setup for local development.

Milestone 2: Backend API Development using Flask & SQLite

- **Activity 2.1**: Develop RESTful API endpoints for photo upload, face clustering, chatbot communication, album mapping, and library analytics.
- **Activity 2.2**: Define SQLAlchemy ORM database models in models/ for **User**, **Photo**, **Face**, **Person**, **Album**, **DeliveryHistory**, and **AgentLog**.
- **Activity 2.3**: Implement database initialization with SQLAlchemy ORM and ensure schema consistency for SQLite.
- **Activity 2.4**: Create modular blueprints in routes/ for core routes: **photos.py**, **faces.py**, **chat.py**, **share.py**, **albums.py**, and **analytics.py**.
- **Activity 2.5**: Implement user authentication using secure JWT tokens (**PyJWT**) and secure password hashing using **Flask-Bcrypt**.

Milestone 3: AI, Vector Search, and Service Integration

- **Activity 3.1**: Integrate **DeepFace** (using the **RetinaFace detector backend** and **Facenet512 embedding model**) alongside **OpenCV** for face detection, matching, and cropping.
- **Activity 3.2**: Configure the **ChromaDB vector database** and develop services in services/vector_service.py for semantic embedding generation and indexing.
- **Activity 3.3**: Build the conversational AI assistant workflow using **LangGraph state machines** in workflows/agent_workflow.py with specialized nodes: **Orchestrator**, **Search**, **Memory**, **Album**, **Sharing**, and **Vision Agents** powered by **Groq Llama 3.3**.
- **Activity 3.4**: Integrate **Gmail SMTP secure mailer** and **Twilio WhatsApp API** services to enable automated photo and album deliveries.
- **Activity 3.5**: Implement background pipelines for unsupervised **DBSCAN face clustering** and color-histogram scene similarity analysis.

Milestone 4: React.js Frontend Development with Vanilla CSS

- **Activity 4.1**: Design a responsive and premium user interface with React.js using modern **Vanilla CSS** with HSL variables, glassmorphic headers, and grid layouts.
- **Activity 4.2**: Implement the frontend **Voice Search Engine** using the browser **Web Speech API** supporting English, Hindi, Spanish, and Telugu.
- **Activity 4.3**: Build interactive UI components for photo upload, face labeling grids, custom album creation, and gallery masonry grids.
- **Activity 4.4**: Develop the AI chatbot interface featuring chat clearing and automated gallery redirection actions (**FILTER_GALLERY_BY_LABEL**).
- **Activity 4.5**: Integrate dashboard metrics with radial progress rings for storage usage and monthly upload charts, and add a library **ZIP backup utility**

Milestone 5: Testing, Validation, and Deployment

- **Activity 5.1**: Conduct backend unit testing for face detection, labeling, and chat services using Python test scripts.
- **Activity 5.2**: Test frontend UI responsiveness, component rendering, voice search flows, and audio clearances.
- **Activity 5.3**: Validate backend–frontend integration and ensure accurate data synchronization, token handling, and error logging.
- **Activity 5.4**: Perform output validation, delivery tracking checks, and storage alert verification.
- **Activity 5.5**: Compile production frontend builds, launch the Flask API backend in production mode, and configure servers to serve both layers securely.

Milestone 1: Environment Setup and Project Initialization

Activity 1.1: Create and activate a Python virtual environment inside the drishyamitra-backend folder.

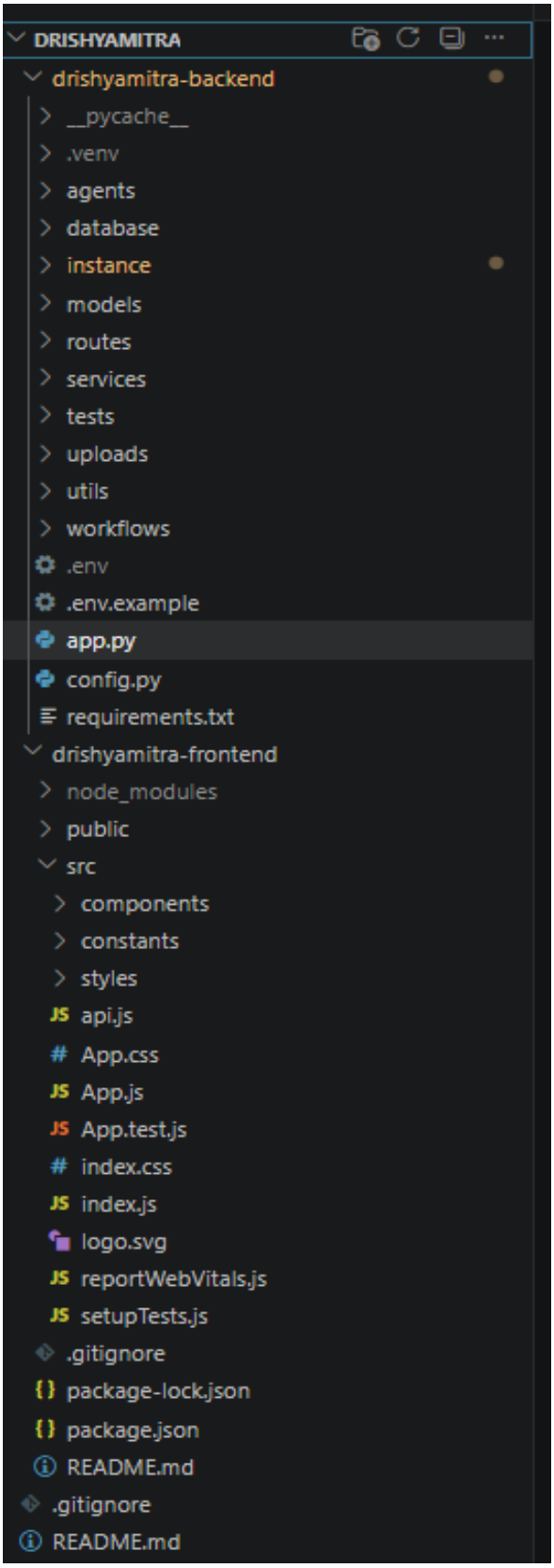
To begin the backend setup, navigate to the drishyamitra-backend subdirectory. Install **Python 3.10 or later** on your system and verify the installation using python --version. Next, create an isolated virtual environment by running the command python -m venv .venv. This ensures that all AI dependencies—such as **DeepFace**, **ChromaDB**, and **LangGraph**—are stored separately from your global Python installation to prevent version conflicts. Once created, activate the environment using .venv\Scripts\activate on Windows or source .venv/bin/activate on macOS/Linux.
Here is the exact terminal command sequence you should run on Windows:

```
PS C:\Drishyamitra> cd drishyamitra-backend
PS C:\Drishyamitra\drishyamitra-backend> python -m venv .venv
PS C:\Drishyamitra\drishyamitra-backend> .venv\Scripts\activate
(.venv) PS C:\Drishyamitra\drishyamitra-backend>
```

Inside this environment, you can install and manage backend dependencies without affecting other applications. It is also recommended to include the .venv folder in your .gitignore file to prevent committing environment packages to version control.

Activity 1.2: Set up the project folder structure for backend (Flask) and frontend (React.js).

Organize the project into two main directories: drishyamitra-backend for the Flask API and drishyamitra-frontend for the React.js client. Inside the backend folder, create subdirectories such as agents/ (for LangGraph agent nodes), workflows/ (for state-machine execution), models/ (for SQLAlchemy ORM definitions), routes/ (for Flask blueprints), and services/ (for face recognition, OpenAI Vision, and vector search). Include app.py as the main entry point and requirements.txt for Python dependencies. For the frontend, maintain src/components/ for modular React page elements, src/styles/ for Vanilla CSS styling sheets, and src/api.js for centralizing Axios requests.


DRISHYAMITRA
drishyamitra-backend
__pycache__
.venv
agents
database
instance
models
routes
services
tests
uploads
utils
workflows
.env
.env.example
app.py
config.py
requirements.txt
drishyamitra-frontend
node_modules
public
src
components
constants
styles
api.js
App.css
App.js
App.test.js
index.css
index.js
logo.svg
reportWebVitals.js
setupTests.js
.gitignore
package-lock.json
package.json
README.md
.gitignore
README.md

```python
import os
from flask import Flask, jsonify
from flask_cors import CORS
from dotenv import load_dotenv

# ---------------------------------------------------------------------------
# Load .env BEFORE anything references os.environ
# ---------------------------------------------------------------------------
load_dotenv()


def create_app(test_config=None):
    """Application factory — builds and configures the Flask app."""

    app = Flask(__name__)

    # ── Configuration ──────────────────────────────────────────────────────
    from config import Config
    app.config.from_object(Config)
    if test_config is not None:
        app.config.from_mapping(test_config)

    # ── CORS — allow React dev server (port 3000) ──────────────────────────
    CORS(app, resources={r"/api/*": {"origins": "*"}})

    # ── Database ───────────────────────────────────────────────────────────
    from database.db import db
    db.init_app(app)

    # ── Ensure upload directory exists ─────────────────────────────────────
    upload_dir = app.config.get("UPLOAD_FOLDER", "uploads")
    os.makedirs(upload_dir, exist_ok=True)

    # ── Register Blueprints ────────────────────────────────────────────────
    from routes.auth import bp as auth_bp
    from routes.photos import bp as photos_bp
    from routes.faces import bp as faces_bp
    from routes.albums import bp as albums_bp
    from routes.chat import bp as chat_bp
    from routes.analytics import bp as analytics_bp
    from routes.share import bp as share_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(photos_bp)
    app.register_blueprint(faces_bp)
    app.register_blueprint(albums_bp)
    app.register_blueprint(chat_bp)
    app.register_blueprint(analytics_bp)
    app.register_blueprint(share_bp)
```

```python
# ── Create tables & seed defaults ──────────────────────────────
with app.app_context():
    # Import all models so SQLAlchemy knows about them
    import models  # noqa: F401
    _run_db_migrations(app)
    db.create_all()
    _seed_default_user(db)
    _backfill_legacy_user_ids(db)

# ── Health-check route ─────────────────────────────────────────
@app.route("/api/health")
def health():
    return jsonify({"status": "ok", "service": "drishyamitra-backend"})

# ── Error handlers ─────────────────────────────────────────────
@app.errorhandler(404)
def not_found(e):
    return jsonify({"error": "Resource not found"}), 404

@app.errorhandler(500)
def internal_error(e):
    return jsonify({"error": "Internal server error"}), 500

return app
```

```python
if __name__ == "__main__":
    app = create_app()
    print("\n[INFO] Drishyamitra Backend running on http://localhost:5000")
    print("[INFO] Photo management API ready\n")
    app.run(debug=True, host="0.0.0.0", port=5000)
```

Activity 1.3: Configure .env files for both backend and frontend.

To configure the environment settings, create two separate configuration files: a backend .env inside the drishyamitra-backend folder to hold sensitive API keys, database settings, and third-party credentials, and a frontend .env inside the drishyamitra-frontend folder to set the backend connection URL. This separates environment variables from the codebase, ensuring secure operations during deployment.
Here are the templates for both files:

Create this file in the backend root directory to configure the databases, AI models, and email/WhatsApp dispatchers:

```
SECRET_KEY=drishyamitra-super-secret-key-change-in-production
DATABASE_URL=sqlite:///drishyamitra.db
UPLOAD_FOLDER=uploads
MAX_CONTENT_LENGTH=52428800
CHROMA_PERSIST_DIR=./chroma_data

# Groq API Key (required for AI Assistant)
# Get yours free at: https://console.groq.com/keys
GROQ_API_KEY=your_groq_api_key_here.venv\Scripts\activate

# OpenAI API Key (required for AI Vision Auto-Tagging)
# Get yours at: https://platform.openai.com/api-keys
OPENAI_API_KEY=your_openai_api_key_here

# Google OAuth 2.0 (required for Google Login verification)
# Get yours at: https://console.cloud.google.com/apis/credentials
GOOGLE_CLIENT_ID=your_google_client_id_here

# Email sharing & Data Backups (SMTP Setup)
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your_email@gmail.com
MAIL_PASSWORD=your_email_app_password
MAIL_USE_TLS=True
MAIL_DEFAULT_SENDER=no-reply@drishyamitra.com

# WhatsApp sharing (Twilio API) — Optional
# Sign up free at: https://www.twilio.com/try-twilio
TWILIO_ACCOUNT_SID=your_twilio_account_sid
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886
```

.Create this file in the frontend root directory so the React application knows how to communicate with the Flask API:

```
# API endpoint pointing to the local Flask server port
REACT_APP_API_BASE_URL=http://localhost:5000
```

Groq API (for AI Chat Assistant):

- **Purpose**: Powers the **LangGraph state machine** to route conversational user queries to specialized backend agents (**Orchestrator**, **Search**, **Memory**, **Album**, **Sharing**, and **Vision**), execute database tool-calling for real-time photo counts, and structure gallery redirection actions.
- **Model Used**: llama-3.3-70b-versatile (as configured in your backend **config.py**).
- **Get API Key**: https://console.groq.com/keys
- **Setup Notes**:

○ Sign up or log in to your **Groq Console** account.

○ Navigate to **API Keys** in the sidebar.

○ Create a new API key and copy it into your backend **.env** file as:

GROQ_API_KEY=gsk_your_actual_groq_api_key_here

1. Gmail API (for Email Delivery Integration)

- **Purpose**: Enables secure packaging and sending of high-resolution photographs as email attachments directly through the chatbot and sharing dashboard.
- **Setup Steps**:

1. Enable 2-Step Verification on your Google Account settings.

2. Generate a secure 16-character **App Password** under the Security settings tab (select 'Mail' and your device type).

3. Copy the password and paste it into your backend .env file as:

SMTP_SERVER=smtp.gmail.com

SMTP_PORT=587

SMTP_PASSWORD=your_16_char_app_password

2. Twilio API for WhatsApp (for WhatsApp Delivery Integration)

- **Purpose**: Dispatches photos and sharing album links automatically to WhatsApp phone numbers through the assistant.

- **Setup Steps**:

1. Register for a free developer account on twilio.com.

2. Access your sandbox details and connect your mobile device by sending the sandbox activation code to the Twilio number.

3. Copy your account credentials and paste them into your backend **.env** file as:

TWILIO_ACCOUNT_SID=ACyour_twilio_account_sid

TWILIO_AUTH_TOKEN=your_twilio_auth_token

3. DeepFace Models (RetinaFace & Facenet512)

- **Purpose**: Locates and crops facial regions within photos, extracts unique face embeddings, and calculates similarities for clustering.

- **Library**: https://github.com/serengil/deepface

- **Setup Steps**:

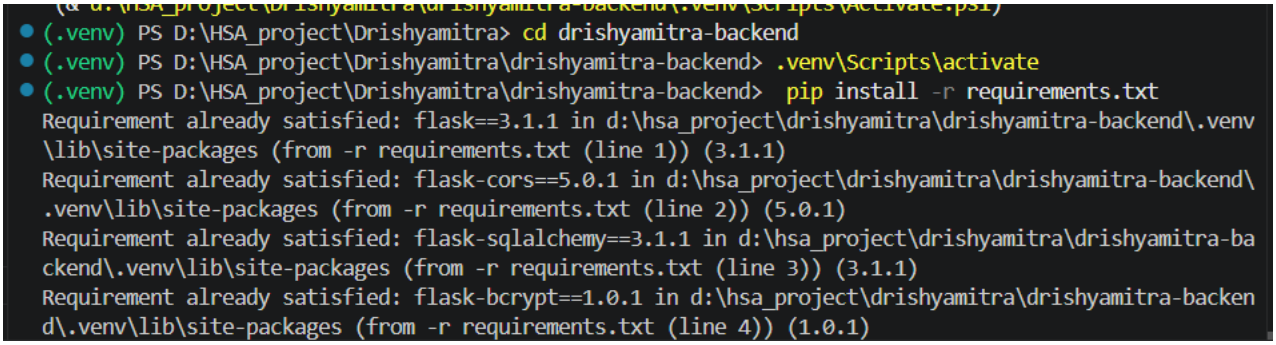
1. Verify that the dependencies are installed by running: pip install -r requirements.txt (which installs deepface==0.0.93).

2. On the first run, the system will automatically download the required detector and embedding model files from the web (no API key needed).

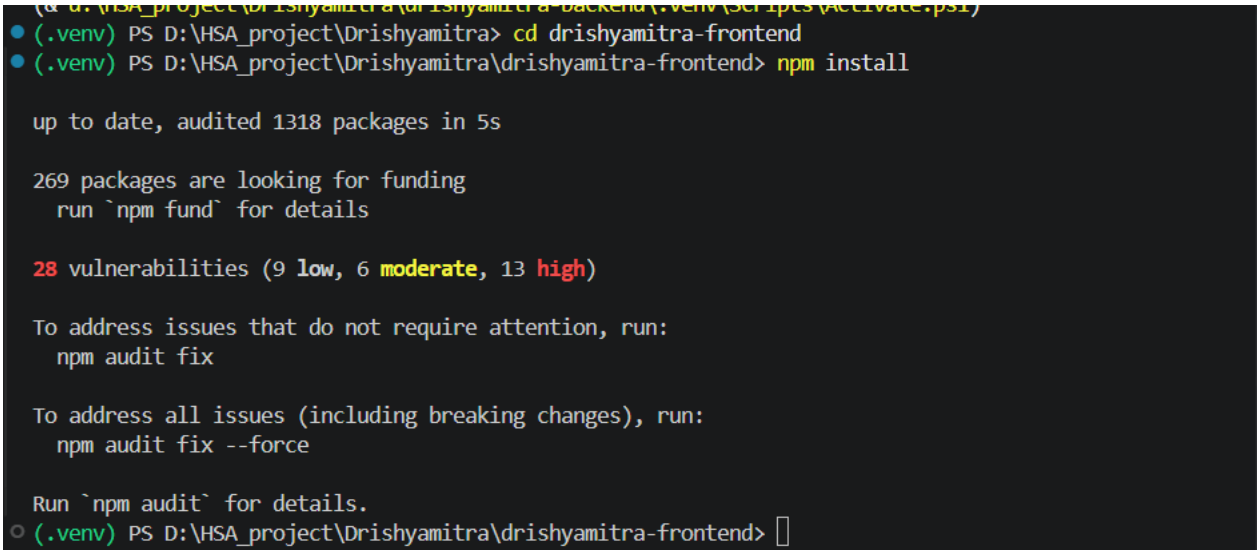
3. Ensure image preprocessing drivers are loaded via the headless OpenCV library: pip install opencv-python-headless.

Activity 1.4: Install dependencies using pip install -r requirements.txt and npm install.

Once the environment and structure are ready, install all required backend dependencies using the command pip install -r requirements.txt. This file lists the key packages for the project, including **Flask**, **SQLAlchemy**, **DeepFace** (for computer vision), **ChromaDB** (for vector embeddings), **LangGraph & Groq** (for agent coordination), **Scikit-Learn** (for clustering), and **Flask-CORS**. Each installation should occur inside the virtual environment to ensure isolation. After installation, verify by running pip list to confirm that all packages are correctly installed.



For the frontend, navigate to the drishyamitra-frontend directory and run the command npm install to install React and related dependencies, such as **Axios** for API calls and **React-Scripts**. Note that styling in Drishyamitra is managed using a modern, customized **Vanilla CSS** design system located inside src/styles/ rather than utility frameworks like Tailwind CSS, giving you maximum control over transitions and layouts. Keep both dependency files (**requirements.txt** and **package.json**) updated for future reproducibility.



Activity 1.5: Verify environment setup and ensure reproducible builds using isolated virtual environments and consistent npm installs.

After installing all packages, verify that both the backend and frontend environments are properly configured and communicating. First, start the Flask development server by running the command python app.py (which serves as the main entry point in Drishyamitra instead of legacy scripts like start_server.py) inside your active virtual environment. Verify that the server establishes the SQLite database, checks migrations, and starts without errors on http://localhost:5000.

```
(.venv) PS D:\HSA_project\Drishyamitra\drishyamitra-backend> python app.py

[INFO] Drishyamitra Backend running on http://localhost:5000
[INFO] Photo management API ready

 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSG
I server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.11:5000
Press CTRL+C to quit
 * Restarting with stat

[INFO] Drishyamitra Backend running on http://localhost:5000
[INFO] Photo management API ready

 * Debugger is active!
 * Debugger PIN: 674-187-323
```

Next, open another terminal, navigate to the drishyamitra-frontend directory, and launch the React client using the command npm start. This compiles the JavaScript bundles and opens the user interface at http://localhost:3000. Verify that the frontend can successfully log in using the seeded credentials (**admin@example.com / password123**) and connect to the API base URL.

```
(.venv) PS D:\HSA_project\Drishyamitra\drishyamitra-frontend> npm start

> drishyamitra-frontend@0.1.0 start
> react-scripts start

(node:4856) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: 'onAfterSetupMiddl
eware' option is deprecated. Please use the 'setupMiddlewares' option.
(Use `node --trace-deprecation ...` to show where the warning was created)
(node:4856) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning: 'onBeforeSetupMid
dleware' option is deprecated. Please use the 'setupMiddlewares' option.
Starting the development server...
Compiled successfully!

You can now view drishyamitra-frontend in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.56.1:3000
```

To ensure reproducible builds across environments, maintain matching dependency versions. Developers should execute pip freeze > requirements.txt in the backend and use the package-lock.json file with npm ci in the frontend to guarantee consistent package installations and prevent runtime version mismatches.

Milestone 2: Backend API Development using Flask

Activity 2.1: Develop RESTful API endpoints for photo management, face recognition, and chatbot functionalities.

Begin by initializing the Flask application and setting up core extensions such as **Flask-CORS** for cross-origin resource sharing and **Flask-SQLAlchemy** for database communication. Instead of legacy extensions like Flask-RESTful or Flask-JWT-Extended, Drishyamitra relies on modular **Flask Blueprints** to isolate routing layers and implements a custom @token_required decorator using **PyJWT** (**PyJWT**) to secure API endpoints. The goal is to build clear RESTful endpoints that map directlytoservices,including:

- /api/photos/upload for Cloudinary image uploads and indexing.
- /api/faces/detect and /api/faces/label for face extraction and identity tagging.
- /api/chat/ for routing queries through the LangGraph state machine.
- /api/analytics/dashboard for fetching live library activity metrics.

```python
from flask import Flask, jsonify, request
from flask_cors import CORS
from config import Config
from models.database import init_db, db
import os
from datetime import datetime
from flask_migrate import Migrate
from dotenv import load_dotenv
from routes.face_routes import face_bp


load_dotenv()


def create_app():
    """Application factory function"""
    app = Flask(__name__)
    app.config.from_object(Config)

    # Initialize extensions FIRST
    CORS(app, resources={r"/api/*": {"origins": "*"}})
    init_db(app)
    Config.init_app(app)
    migrate = Migrate(app, db)

    # Register blueprints - ALL imports here
    from routes.auth_routes import auth_bp
    from routes.chat_routes import chat_bp
    from routes.photo_routes import photo_bp
    from routes.delivery_routes import delivery_bp
    from routes.history_routes import history_bp
    from routes.edit_routes import edit_bp

    # Register all blueprints
    app.register_blueprint(auth_bp, url_prefix='/api/auth')
    app.register_blueprint(chat_bp, url_prefix='/api/chat')
    app.register_blueprint(photo_bp, url_prefix='/api/photos')
    app.register_blueprint(delivery_bp, url_prefix='/api/delivery')
    app.register_blueprint(history_bp, url_prefix='/api/history')
    app.register_blueprint(edit_bp, url_prefix='/api/edit')
    app.register_blueprint(face_bp, url_prefix='/api/face')
```

Each route is decoupled, returning structured JSON payloads and standard HTTP status codes. Implement robust error handlers within the application factory to gracefully return JSON error responses for 404 Not Found and 500 Internal Server Error occurrences, preventing HTML stack-trace leaks.

Activity 2.2: Define SQLAlchemy ORM models for User, Photo, Face, Person, and DeliveryHistory.

Design the data layer using **SQLAlchemy ORM** to map Python classes to relational database tables. In Drishyamitra, define the database models inside the models/ folder:

- **User**: Manages profiles and stores credentials (username, email, password_hash, role).
- **Photo**: Stores photo metadata, Cloudinary file paths, visual features, descriptions, tags, and favorite flags.
- **Face**: Represents detected faces with coordinates (bounding boxes), 512-dimensional embeddings, and label details.
- **Person**: Groups face clusters and links them to users (storing custom color tags and emojis).
- **Album**: Tracks manual and auto-categorized collections (e.g., Weddings, Birthdays).
- **DeliveryHistory**: Logs photo sharing dispatches, status codes, and error messages.
- **AgentLog**: Saves conversation logs for the chatbot history. Establish relationships with cascading delete settings (e.g., deleting a User deletes their associated Photos; deleting a Photo removes its associated Face records), ensuring strict referential integrity.

Activity 2.3: Implement database initialization with db.create_all() and ensure schema consistency.

Initialize the database connection using **Flask-SQLAlchemy** inside **app.py** and **config.py**. The configuration parses the **DATABASE_URL** environment variable, defaulting to local SQLite (**sqlite:///drishyamitra.db**). Instead of standard Alembic migrations, Drishyamitra runs a custom database schema check function (_run_db_migrations) inside the application context on startup. This function automatically scans the active database file, checks for table structures, and injects missing columns (such as user_id or background_features) before executing db.create_all() to initialize new tables. This prevents data loss, aligns schemas across systems, and seeds default event albums and an admin user.

Activity 2.4: Create modular routes for core services — Photo Upload, Face Detection, Chat Assistant, and Share Integration.

Encapsulate core API pathways into modular **Flask Blueprints** inside the **routes/** directory:

- **auth.py**: Manages registration, logins, profiles, and backup downloads.
- **photos.py**: Manages multi-file uploads to Cloudinary, handles database registration, and routes searches.
- **faces.py**: Triggers the DeepFace detection pipeline, allows face labeling, and runs background DBSCAN clustering tasks.
- **chat.py**: Serves as the AI assistant gateway, routing prompts through the LangGraph agents.
- **share.py**: Coordinates photo packaging and transfers to external communication interfaces. By registering these modular blueprints in **app.py**, you achieve a clean separation of concerns and allow easy scalability for additional services.

Activity 2.5: Implement authentication using token-based access and secure password hashing.

Establish security boundaries by encrypting user passwords and securing backend routes. During registration and logins, user passwords are encrypted using **Flask-Bcrypt** before they are stored in the database. When a user logs in, the backend generates a signed JSON Web Token (JWT) using the **PyJWT** library, which includes the user ID and expiration settings. Routes are protected using a custom @token_required decorator located in **utils/auth_helpers.py**. This decorator intercepts requests, extracts the token from the Authorization header, validates the signature, and rejects unauthorized requests with a 401 Unauthorized response.

Milestone 3: AI, Vector Search, and Service Integration

Activity 3.1: Integrate DeepFace models (Facenet512, RetinaFace) for face detection and recognition.

Begin by configuring the **DeepFace** library to handle image-based facial analysis. Drishyamitra utilizes the Facenet512 model to generate 512-dimensional numerical vectors that uniquely represent facial structures, enabling high-accuracy matching. It implements the RetinaFace detector backend, which provides superior performance in locating faces under varying illumination, head angles, and partial occlusions. Each uploaded image is processed through this vision pipeline to extract bounding boxes, confidence scores, and embeddings.
Here is the actual Python class structure used in your project's **services/vision_service.py**:

```python
import logging

logger = logging.getLogger(__name__)


class VisionService:
    """Handles face detection and embedding generation using DeepFace."""

    DETECTOR_BACKEND = 'retinaface'
    MODEL_NAME = 'Facenet512'

    @staticmethod
    def detect_faces(image_path):
        """
        Detect faces in an image and generate 512-dimensional embeddings.

        Uses DeepFace with the RetinaFace detector for robust face localisation
        and Facenet512 for high-quality 512-dim embedding vectors.

        Args:
            image_path (str): Absolute filesystem path to the image file.

        Returns:
            list[dict]: A list of dicts, each containing:
                - bounding_box (dict): ``{"x": int, "y": int, "w": int, "h": int}``
                - embedding (list[float]): 512-dimensional face embedding.
                - confidence (float): Detection confidence score (0-1).
              Returns an empty list when no faces are found or on any error.
        """
        try:
            from deepface import DeepFace
        except ImportError:
            logger.warning(
                "deepface is not installed — face detection unavailable. "
                "Install with: pip install deepface"
            )
            return []

        try:
            results = DeepFace.represent(
                img_path=image_path,
                model_name=VisionService.MODEL_NAME,
                detector_backend=VisionService.DETECTOR_BACKEND,
                enforce_detection=False,
```

```
            enforce_detection=False,
        )
    except Exception as exc:
        logger.error("DeepFace.represent failed for '%s': %s", image_path, exc)
        return []

    if not results:
        return []

    faces = []
    for entry in results:
        facial_area = entry.get('facial_area', {})
        embedding = entry.get('embedding', [])
        confidence = entry.get('face_confidence', 0.0)

        # Skip entries with trivially small or missing bounding boxes
        if not facial_area or not embedding:
            continue

        bounding_box = {
            'x': int(facial_area.get('x', 0)),
            'y': int(facial_area.get('y', 0)),
            'w': int(facial_area.get('w', 0)),
            'h': int(facial_area.get('h', 0)),
        }

        # Skip detections that are clearly noise (zero-area boxes)
        if bounding_box['w'] <= 0 or bounding_box['h'] <= 0:
            continue

        faces.append({
            'bounding_box': bounding_box,
            'embedding': embedding,
            'confidence': float(confidence),
        })

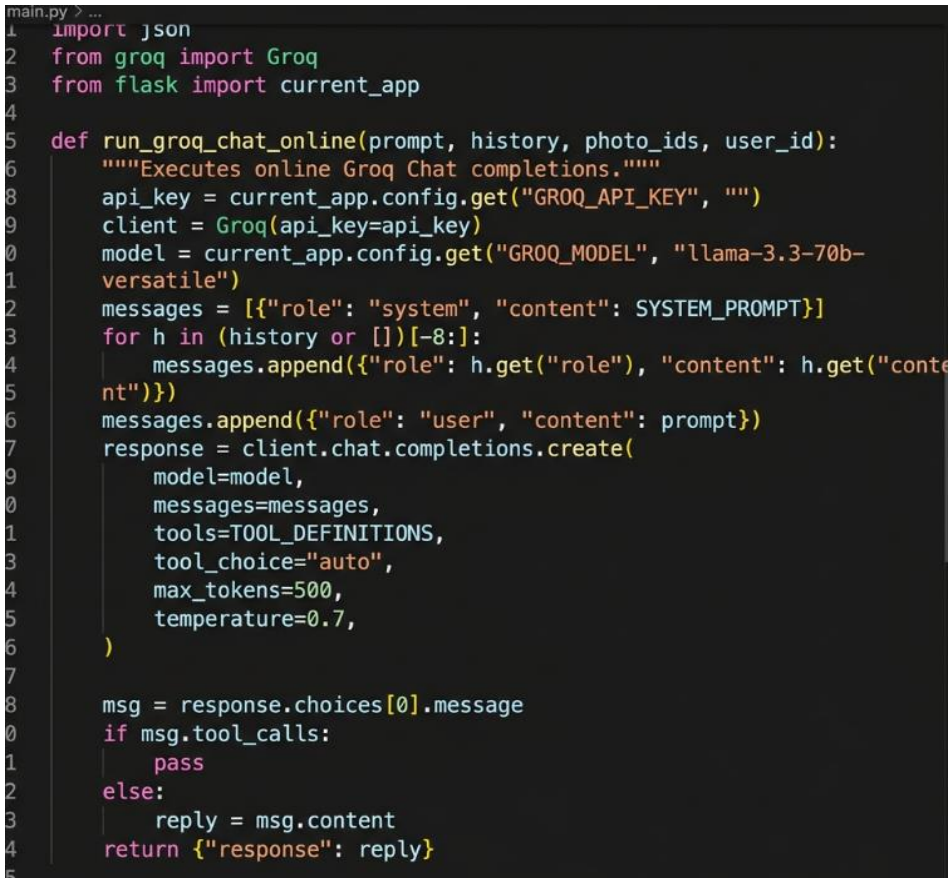
    logger.info(
        "Detected %d face(s) in '%s'.",
        len(faces),
        image_path,
    )
    return faces
```

Rather than using a local pickle file, the generated embeddings are saved directly in the SQLite database under the Face table. When new photos are uploaded, the system queries known face records and calculates similarity scores using **Cosine Similarity** via **scikit-learn** to automatically assign names to recurring individuals. If the face is unrecognized, it is stored as NULL and dynamically categorized into placeholder clusters in the background.

.

Activity 3.2: Develop the AI Chat Assistant using Groq API and LangGraph for agent orchestration.

Leverage the **Groq API** running the **Llama 3.3 70B** model (llama-3.3-70b-versatile) to build an intelligent conversational interface. Rather than running a single linear script, Drishyamitra compiles a state machine using **LangGraph** to construct an agent-based architecture. A central **Orchestrator Agent** analyzes user query syntax and routes the conversation state dynamically to specialized nodes: **Search Agent** (for querying database contents), **Memory Agent** (for resolving person profiles), **Album Agent** (for grouping files), **Sharing Agent** (for packaging attachments), or **Vision Agent** (for detailed image queries).
Here is the actual structure of the direct Groq integration helper with tool calling inside your project's **routes/chat.py**:



The system includes pre-defined tool parameters allowing the LLM to trigger real-time database functions: **get_system_counts** (to count library items), **search_photos** (to search text/vectors), **get_person** (to fetch face files), and **create_album** (to initialize new albums). A three-tier execution hierarchy guarantees reliability: the server first runs the **LangGraph workflow**, falls back to direct **Groq completion** if routing fails, and executes an offline, regex-based database processor if no internet connection is available.

Activity 3.3: Implement Email Delivery using SMTP integration for photo sharing.

Configure backend SMTP services to enable secure and automated email-based photo sharing. Authenticate using a secure Google 16-character **App Password** and your Gmail address, bypassing direct OAuth token caching models. Create a robust utility function that composes messages using **MIMEMultipart**, attaches dynamic text, and loops over file paths to attach images as **MIMEImage** structures. To prevent SMTP transmission timeouts and payload restrictions, integrate automatic image scaling and compression using **PIL (Pillow)** for attachments exceeding 500 KB. Here is the code structure of your project's email worker located inside **services/sharing_service.py**:

```python
import os
import logging
import smtplib
import requests
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.image import MIMEImage
from datetime import datetime, timezone

from database.db import db
from models.sharing import DeliveryHistory

logger = logging.getLogger(__name__)


def _send_real_email(recipient, person_name, photo_paths):
    """Send an email with photo attachments via SMTP/Gmail."""
    smtp_user = os.environ.get("SMTP_USER")
    smtp_password = os.environ.get("SMTP_PASSWORD")
    smtp_server = os.environ.get("SMTP_SERVER", "smtp.gmail.com")
    smtp_port_str = os.environ.get("SMTP_PORT", "465")

    if not smtp_user or not smtp_password:
        logger.warning("[SMTP] SMTP_USER or SMTP_PASSWORD not set in .env. Skipping real email send.")
        return False, "SMTP credentials SMTP_USER or SMTP_PASSWORD not configured in .env file"

    try:
        primary_port = int(smtp_port_str)
    except ValueError:
        primary_port = 465

    fallback_port = 587 if primary_port == 465 else 465

    msg = MIMEMultipart()
    msg['From'] = smtp_user
    msg['To'] = recipient
    msg['Subject'] = f"Drishyamitra — Photos shared with you"

    body_text = f"Hi,\n\nHere are the photos of '{person_name or 'selected collection'}' shared via Drishyamitra AI Assistant.\n\nEnjoy!"
    msg.attach(MIMEText(body_text, 'plain'))

    # Attach images securely as MIMEImage (compressing large files dynamically to prevent SMTP disconnects)
    from PIL import Image
    import io

    for path in photo_paths:
        if os.path.exists(path):
            try:
                filename = os.path.basename(path)
                file_size = os.path.getsize(path)

                # Compress if file size exceeds 500 KB to keep total email payload light
                if file_size > 500 * 1024:
```

To prevent API request blockages, the email execution is delegated to an asynchronous background worker using **threading.Thread**. The status of each delivery (e.g., pending, delivered, or failed with detailed error logs) is saved directly in the SQLite **DeliveryHistory** table, allowing users to track shared folders and activity summaries from the Live Analytics Dashboard.

Activity 3.4: Integrate Twilio API for WhatsApp for automated photo delivery

Configure backend media delivery services to enable automated photo sharing via WhatsApp. The system integrates the official Twilio API for WhatsApp Sandbox using your Twilio Account SID and Auth Token credentials stored securely in the environment. Because Twilio requires public HTTP endpoints to retrieve and transmit attachments, develop a helper function that uploads local photos to the Litterbox API for temporary 1-hour hosting. Once public URLs are generated, format the recipient's phone number, append the whatsapp: scheme, and post the payload containing the media link to Twilio's servers.
Here is the code structure of your WhatsApp delivery worker located inside services/sharing_service.py:

```python
import os
import requests

def _send_real_whatsapp(recipient, person_name, photo_paths):
    account_sid = os.environ.get("TWILIO_ACCOUNT_SID")
    auth_token = os.environ.get("TWILIO_AUTH_TOKEN")
    from_whatsapp = os.environ.get("TWILIO_WHATSAPP_FROM", "whatsapp:+14155238886")
    to_number = recipient
    if not to_number.startswith("whatsapp:"):
        clean_num = "".join(c for c in to_number if c.isdigit() or c == '+')
        if not clean_num.startswith("+"):
            clean_num = "+91" + clean_num if len(clean_num) == 10 else "+" + clean_num
        to_number = f"whatsapp:{clean_num}"

    media_urls = []
    for path in photo_paths[:10]:
        public_url = _upload_for_public_url(path) if not path.startswith("http") else pat
        if public_url:
            media_urls.append(public_url)

    twilio_url = f"https://api.twilio.com/2010-04-01/Accounts/{account_sid}/Messages.json
    errors = []
    for idx, m_url in enumerate(media_urls):
        res = requests.post(
            twilio_url,
            auth=(account_sid, auth_token),
            data={
                "From": from_whatsapp,
                "To": to_number,
                "Body": f"shared via Drishyamitra - Photo {idx+1}",
                "MediaUrl": m_url,
            },
            timeout=15
        )
        if res.status_code not in (200, 201):
            errors.append(res.text)

    return (True, None) if not errors else (False, "; ".join(errors))
```

To optimize performance, wrap the execution inside an asynchronous thread in **SharingService.send_whatsapp**. The system logs delivery statuses, intercepts Twilio-specific API codes (such as 63016 for expired sandbox sessions or 63019 for media download errors), and records transactions in the SQLite **DeliveryHistory** model for transparent tracking on the Live Analytics Dashboard.

Activity 3.5: Build background services for batch processing, folder organization, and delivery history tracking.

To maintain high responsiveness without adding complex external overhead like Celery and Redis, Drishyamitra utilizes Python's native **threading** module to handle asynchronous background processes. Spawning background threads allows the server to instantly return a response to the frontend client while delegating long-running tasks to non-blocking threads:

- **Asynchronous Sharing Dispatch**: Triggers separate daemon threads to execute SMTP mail handshakes and post image payloads to Twilio's WhatsApp servers asynchronously.
- **Automatic DBSCAN Face Clustering**: Automatically schedules the DBSCAN background grouping algorithm (run_face_clustering) in **routes/faces.py** upon photo upload, mapping new faces to person IDs in the background.
- **Automated Scene & Places Album Assignment**: Launches background calculations in **services/scene_clustering_service.py** to compute background color histograms and group photos into event albums (Weddings, Birthdays, Anniversaries).

All actions are tracked by the **DeliveryHistory** model. This service records delivery mediums (SMTP, Twilio), timestamps, and error codes directly in the SQLite database, feeding live analytics metrics to the user dashboard for transparent operation tracking.

Milestone 4: React.js Frontend Development

Activity 4.1: Design a responsive and premium UI with React.js using custom design tokens.

Begin by structuring the React.js frontend to ensure modularity, responsiveness, and visual excellence. Instead of using generic Tailwind CSS utilities, Drishyamitra implements a specialized design system based on **Google Photos (GP) style tokens** defined in **src/styles/theme.js**. This setup integrates Google Sans typography, customized slate-obsidian surface colors, custom card elevation shadows, and keyframe micro-animations (e.g., fadeUp and shimmer) to provide smooth visual feedback. The layout features a responsive dashboard-style interface with a collapsible navigation sidebar, an interactive voice-enabled search bar, and flexible masonry grids.

Here is the code structure of your project's styling tokens defined in **src/styles/theme.js**:

```
export const GP = {
  // Base surfaces — Slate Obsidian
  white: "#ffffff",
  surface: "#f8fafc",              // Slate 50 — main page background
  surfaceElevated: "#ffffff",
  containerDark: "#0f172a",        // Slate 900 — dark mode containers / sidebar
  containerMid: "#1e293b",         // Slate 800 — section headers, cards on dark bg

  // Borders
  border: "#e2e8f0",               // Slate 200
  borderLight: "#f1f5f9",          // Slate 100

  // Typography
  textPrimary: "#1e293b",          // Slate 800 — primary headers & body
  textSecondary: "#475569",        // Slate 600
  textTertiary: "#94a3b8",         // Slate 400

  // Accent — Cyan (replaces old blue)
  blue: "#06b6d4",                 // Cyan 500 — primary actions, badges, links
  blueLight: "#e0f7fa",            // Cyan 50 — hover fills, chip backgrounds
  blueDark: "#0891b2",             // Cyan 600 — pressed / hover states

  // Supporting palette
  teal: "#14b8a6",                 // Teal 500
  tealLight: "#ccfbf1",            // Teal 100
  coral: "#f43f5e",                // Rose 500
  coralLight: "#ffe4e6",           // Rose 100
  amber: "#f59e0b",                // Amber 500
  amberLight: "#fef3c7",           // Amber 100
  green: "#22c55e",                // Green 500
  greenLight: "#dcfce7",           // Green 100
  purple: "#a855f7",               // Purple 500
  purpleLight: "#f3e8ff",          // Purple 100

  // Shadows
  shadow1: "0 1px 2px rgba(15,23,42,0.08), 0 1px 3px rgba(15,23,42,0.06)",
  shadow2: "0 2px 6px rgba(15,23,42,0.12), 0 1px 2px rgba(15,23,42,0.08)",
  shadow3: "0 4px 12px rgba(15,23,42,0.12), 0 2px 4px rgba(15,23,42,0.08)",
  shadowHover: "0 8px 24px rgba(15,23,42,0.16), 0 2px 8px rgba(15,23,42,0.10)",
};
```

This centralized design system injects custom styles globally across all components, standardizing hover effects (e.g., photo overlays, chip highlights, and sidebar selections). This approach decouples presentation tokens from structural logic, ensuring that the app remains visually consistent and easily customizable in the future.

Activity 4.2: Implement photo upload, face labeling, and gallery viewing components with state management.

Develop the Photo Upload Component inside **UploadModal.js** to manage single or bulk file uploads using HTML drag-and-drop APIs, sending payloads to the backend /api/photos/upload endpoint using the Axios client defined in **api.js**. Build a Face Labeling interface inside **PeoplePage.js** that fetches unrecognized face templates and allows users to group, label, and merge clusters.
Create a dynamic Gallery View component inside **GalleryPage.js** to retrieve indexed photos from the database and render them in a responsive masonry layout. Implement horizontal tab-based filters to display "All Photos", "Favourites", "Birthdays", "Weddings", "Anniversaries", or custom albums, alongside a "Select" mode for batch deletes and label associations. Integrate **PhotoDetailModal.js** to display single images in high-resolution, manage favorites, draw cropped face region highlights, and support keyboard arrow-key swipe navigation.

```jsx
import React, { useState, useRef } from "react";
import { api } from "../../api";
import { GP } from "../../styles/theme";
import IconBtn from "../common/IconBtn";
import Spinner from "../common/Spinner";
import ProgressBar from "../common/ProgressBar";

export default function UploadModal({ onClose, onResult }) {
  const [analyzing, setAnalyzing] = useState(false);
  const [result, setResult] = useState(null);
  const [dragging, setDragging] = useState(false);
  const [uploadProgress, setUploadProgress] = useState(null); // { current: 1, total: 3, name: "img.jpg" }
  const fileRef = useRef();



  async function handleMultipleUploads(files) {
    setAnalyzing(true);
    setUploadProgress({ current: 1, total: files.length, name: files[0].name });

    let successCount = 0;
    let failedCount = 0;
    let lastResult = null;

    for (let i = 0; i < files.length; i++) {
      const file = files[i];
      setUploadProgress({ current: i + 1, total: files.length, name: file.name });
      try {
        const res = await api.photos.upload(file);
        successCount++;
        lastResult = {
          name: file.name,
          description: res.description || "Photo processed and indexed.",
          faces: res.persons || [],
          folder: res.folder || "Events",
          tags: res.tags || [],
          confidence: res.analysis?.confidence || 0.95
        };
      } catch (err) {
        console.error(err);
        failedCount++;
      }
    }

    setUploadProgress(null);
    setAnalyzing(false);

    if (successCount > 0) {
      if (files.length === 1 && lastResult) {
        setResult(lastResult);
      } else {
        setResult({
          isSummary: true,
          message: `Successfully uploaded and analyzed ${successCount} photo(s).`,
```

```jsx
import React, { useState, useEffect, useCallback } from "react";
import { GP } from "../../styles/theme";
import IconBtn from "../common/IconBtn";
import { api } from "../../api";

export default function PhotoDetailModal({ photo, onClose, onDelete, onShare, photos, currentIndex, onIndexChange, onReLabel }) {
  const [assignLabelStr, setAssignLabelStr] = useState("");
  const [persons, setPersons] = useState([]);
  const [isAssigning, setIsAssigning] = useState(false);
  const [isMovingAlbum, setIsMovingAlbum] = useState(false);

  useEffect(() => {
    api.faces.persons().then(p => setPersons(p)).catch(console.error);
  }, []);

  const activePhoto = photos && currentIndex !== undefined
    ? (photos[currentIndex] || photos[photos.length - 1] || photo)
    : photo;

  const activePhotoEventAlbum = activePhoto?.album_names?.find(name => ["Birthdays", "Weddings", "Anniversaries"].includes(name)) || "";

  const handleMoveAlbum = async (e) => {
    const targetAlbum = e.target.value;
    if (!activePhoto) return;
    setIsMovingAlbum(true);
    try {
      await api.photos.moveAlbum(activePhoto.id, targetAlbum);
      if (onReLabel) {
        await onReLabel(activePhoto.id, assignLabelStr);
      }
    } catch (err) {
      alert("Failed to move photo to event folder.");
    } finally {
      setIsMovingAlbum(false);
    }
  };

  const handleAssignLabel = async () => {
    if (!assignLabelStr.trim() || !activePhoto) return;
    setIsAssigning(true);
    try {
      const res = await api.photos.assignLabel(activePhoto.id, assignLabelStr);
      setAssignLabelStr("");
      if (onReLabel) {
        await onReLabel(activePhoto.id, assignLabelStr);
      } else {
        alert(res.message);
        if (onClose) onClose();
      }
    } catch (err) {
      alert("Failed to assign label");
    } finally {
      setIsAssigning(false);
    }
  };

  const handlePrev = useCallback(() => {
    if (photos && onIndexChange && currentIndex !== undefined) {
      const prevIdx = (currentIndex - 1 + photos.length) % photos.length;
      onIndexChange(prevIdx);
      setAssignLabelStr("");
    }
  }, [photos, currentIndex, onIndexChange]);

```

Activity 4.3: Develop the AI chatbot interface for natural language photo queries and delivery requests.

Design and implement a full-screen conversational assistant panel inside **ChatPage.js**. The interface features an interactive layout containing a collapsible history sidebar, clickable quick-prompt suggestion chips, real-time typing indicators, and distinct message bubbles for user and AI responses. The component connects to the backend gateway /api/chat/ using the Axios API module defined in **api.js**. To ensure local persistence, serialize conversation lists in the browser's localStorage under the key drishyamitra_conversations.
Here is the code structure of your chatbot send handler located inside **src/components/pages/ChatPage.js**:

```jsx
import React, { useState, useEffect, useRef } from "react";
import { api } from "../../api";
import { GP } from "../../styles/theme";
import PhotoDetailModal from "../gallery/PhotoDetailModal";
import PersonPhotosModal from "../gallery/PersonPhotosModal";

const extractChatAction = (text) => {
  if (!text) return null;

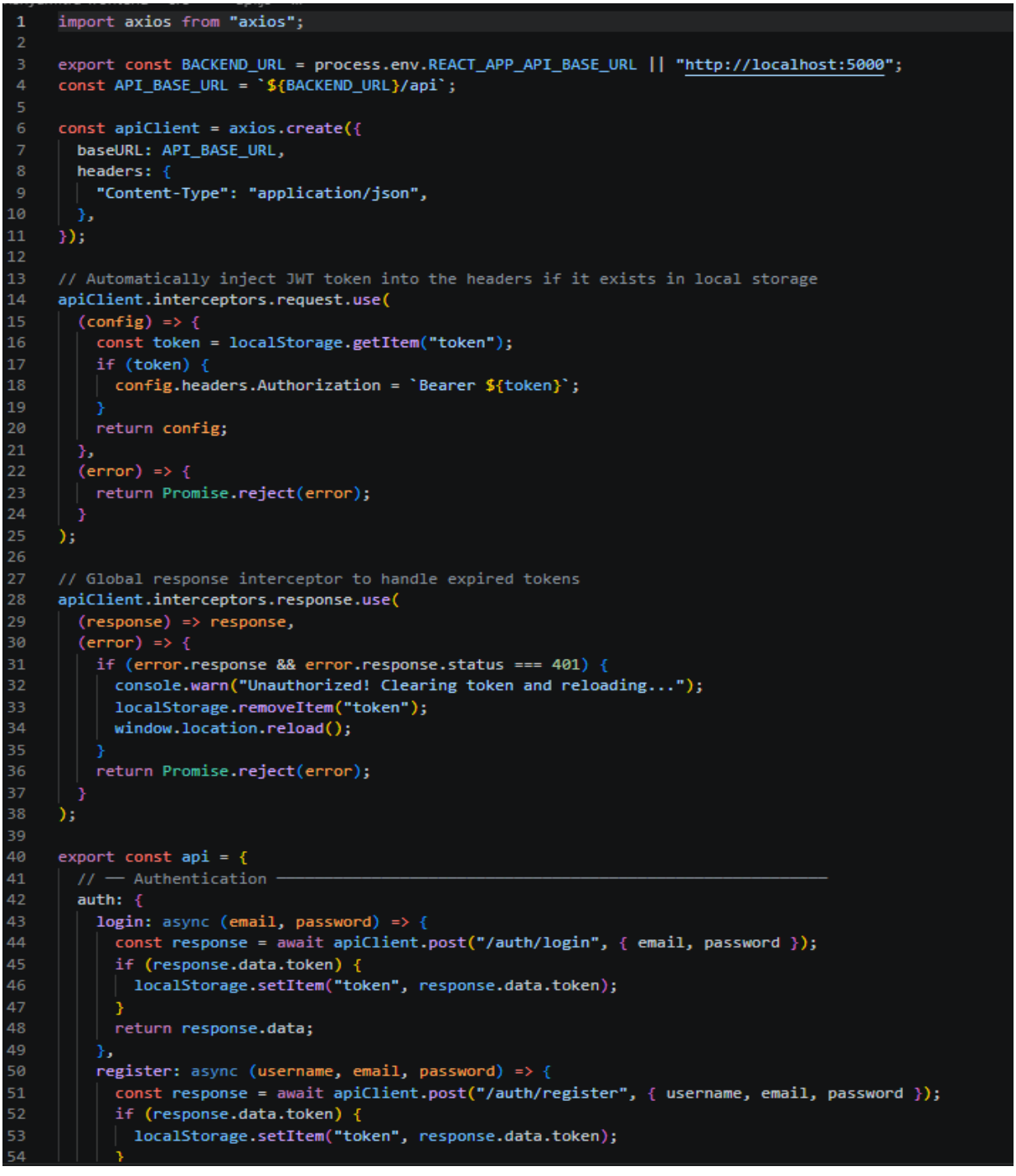
  // 1. Try markdown json block first
  const mdMatch = text.match(/```json\s*([\s\S]*?)\s*```/);
  if (mdMatch) {
    try {
      const parsed = JSON.parse(mdMatch[1]);
      if (parsed && (parsed.action === "EXECUTE_WHATSAPP_SHARE" || parsed.action === "EXECUTE_EMAIL_SHARE" || parsed.action === "FILTER_GALLERY_BY_LABEL")) {
        return parsed;
      }
    } catch (e) {}
  }

  // 2. Try raw JSON by finding '{' containing '"action"' and matching brackets
  const startIndex = text.search(/\{\s*("action"|'action')/);
  if (startIndex !== -1) {
    let braceCount = 0;
    let endIndex = -1;
    for (let i = startIndex; i < text.length; i++) {
      if (text[i] === "{") braceCount++;
      else if (text[i] === "}") {
        braceCount--;
        if (braceCount === 0) {
          endIndex = i;
          break;
        }
      }
    }
    if (endIndex !== -1) {
      const rawJson = text.substring(startIndex, endIndex + 1);
      try {
        const parsed = JSON.parse(rawJson);
        if (parsed && (parsed.action === "EXECUTE_WHATSAPP_SHARE" || parsed.action === "EXECUTE_EMAIL_SHARE" || parsed.action === "FILTER_GALLERY_BY_LABEL")) {
          return parsed;
        }
      } catch (e) {}
    }
  }
  return null;
};
```

Instead of spawning duplicate pages, the interface captures structural instructions. When the user asks to "show photos of Sathish", the parser extracts the query targets and updates the parent component state (setSearch, setPage), transitioning the user to the gallery automatically. The page also features a "Delete History" action which runs an asynchronous call to /api/chat/clear to flush the user's conversational logs from the backend server database.

Activity 4.4: Integrate real-time responses from backend APIs for recognition, search, and chat actions.

Establish communication between the frontend client and the Flask backend using a centralized **Axios Client** inside **src/api.js**. The module reads the connection URL from environment settings, defaulting to local host port 5000 (**http://localhost:5000**). Implement interceptor hooks to automate token security and error routing: a **request interceptor** that injects JWT Bearer tokens from local storage into the headers, and a **response interceptor** that catches 401 Unauthorized codes (expired sessions), flushes invalid tokens, and reloads the viewport.
Here is the actual Axios client and interceptor configuration in your project's **src/api.js**:



```
import axios from "axios";

export const BACKEND_URL = process.env.REACT_APP_API_BASE_URL || "http://localhost:5000";
const API_BASE_URL = `${BACKEND_URL}/api`;

const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    "Content-Type": "application/json",
  },
});

// Automatically inject JWT token into the headers if it exists in local storage
apiClient.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Global response interceptor to handle expired tokens
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      console.warn("Unauthorized! Clearing token and reloading...");
      localStorage.removeItem("token");
      window.location.reload();
    }
    return Promise.reject(error);
  }
);

export const api = {
  // ── Authentication ──────────────────────────────────────────
  auth: {
    login: async (email, password) => {
      const response = await apiClient.post("/auth/login", { email, password });
      if (response.data.token) {
        localStorage.setItem("token", response.data.token);
      }
      return response.data;
    },
    register: async (username, email, password) => {
      const response = await apiClient.post("/auth/register", { username, email, password });
      if (response.data.token) {
        localStorage.setItem("token", response.data.token);
      }
```

Group all backend endpoints inside a single nested **api** object containing subgroups: **api.auth** (profile & backup operations), **api.photos** (file uploads & details), **api.faces** (labeling & clustering triggers), **api.albums** (creation & assignment), **api.chat** (assistant dispatches), and **api.analytics** (live metrics summaries). Use a custom **showNotif** notification helper inside your React components to display success or warning toast blocks on screen, ensuring a consistent user experience.

Activity 4.5: Configure environment variables with REACT_APP_API_BASE_URL and optimize builds for deployment.

Set up the frontend .env file with your environment settings, adding the variable REACT_APP_API_BASE_URL=http://localhost:5000 to link the React application with the Flask API server. Because Drishyamitra's frontend is constructed using **Create React App**, reference this variable dynamically inside your API client **src/api.js** using **process.env**. This ensures the backend connection endpoint remains configurable and easy to modify across development, testing, and production environments without changing the source code.
Before hosting, optimize the frontend by running the command npm run build in the drishyamitra-frontend root directory. This triggers the **Webpack** compiler, which executes code splitting, removes unused codes (tree shaking), minifies JavaScript/CSS files, and compiles your project into a static, production-ready directory named **build/**. Serve these static assets through a production-grade server and verify connectivity to the Flask endpoints to ensure high-performance loading with minimal server overhead.

Milestone 5: Testing and Deployment

Activity 5.1: Activity 5.1: Conduct backend unit testing for face detection, labeling, and chat services using test scripts.

Begin backend testing by executing automated unit tests for each core service inside the **drishyamitra-backend/tests/** directory. Drishyamitra utilizes Python's built-in **unittest** framework. For computer vision testing, **test_vision.py** loads sample image datasets from the **test_assets/** directory to verify face coordinates, confidence scores, and **DeepFace (RetinaFace/Facenet512)** configurations. Use **test_faces.py** to check the background **DBSCAN clustering pipeline** and database transactions.

For the AI Assistant, implement test cases in **test_workflow.py** to simulate chat messages and verify that the **LangGraph state machine** routes intent to the correct agent node. Use the **unittest.mock** library to mock third-party API payloads—such as **Groq**, **OpenAI**, **Gmail SMTP server**, and **Twilio**—enabling you to test sharing dispatches and error reporting logic without making real network calls.

Activity 5.2: Perform frontend testing for UI responsiveness, component rendering, and chat flow validation.

Conduct frontend testing using **Jest** and **React Testing Library** (pre-packaged inside your React environment) to ensure that UI components render correctly. Validate core interfaces—such as **UploadModal.js** file drag-zones, **GalleryPage.js** grids, and the full-page chatbot **ChatPage.js**—across various screen resolutions to verify HSL theme responsiveness. For the chatbot, verify that typing indicators, quick-suggestion chips, and viewport actions (like **FILTER_GALLERY_BY_LABEL**) execute cleanly on screen. Test edge cases, such as token authorization drops or network disconnects, to confirm that response interceptors trigger reloads to block unauthorized access.
To run frontend tests, execute this command in the frontend root: npm test

Activity 5.3: Validate backend–frontend integration and ensure accurate data synchronization between layers.

Integration testing focuses on validating data flows between your React client and your Flask API server. Verify that HTTP requests routed via your Axios client (**src/api.js**) trigger the correct database updates and return clean JSON packages. Test the complete user journey: upload an image via the React UI, check if the Flask backend runs face detection and generates OpenAI Vision descriptions, verify that DBSCAN face-clustering creates a placeholder profile card in the "People" view, and confirm that labeling updates are securely written to the SQLite database.

Activity 5.4: Perform output validation, delivery tracking checks, and user experience review.

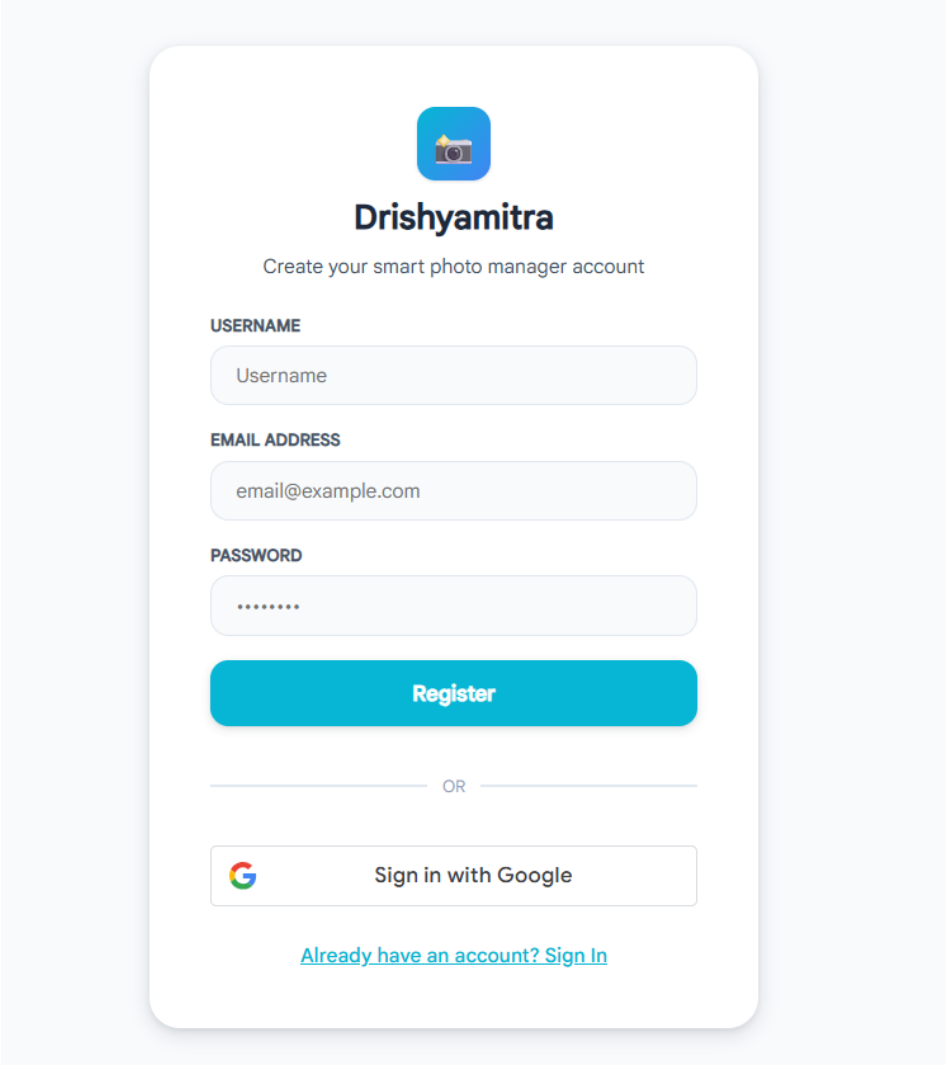
Perform an end-to-end validation of all application outputs to ensure reliability. Verify that the **DeepFace** recognition models correctly identify individuals, that the color-histogram service organizes photos into the correct albums (Weddings, Birthdays), and that all SMTP emails and Twilio WhatsApp dispatches are recorded in the SQLite database under the **DeliveryHistory** model. Check that Twilio sandbox session warnings (error 63016) or attachment constraints (error 63019) are logged, enabling transparent tracking on the Live Analytics Dashboard. Review visual response times and transition animations to refine loading states and ensure a smooth user experience.

Activity 5.5: Deploy production build, configure server variables, and host services.

Prepare the application for production hosting. First, compile the frontend into a minified, static bundle by running the command npm run build inside the drishyamitra-frontend folder, generating a production-ready **build/** directory. Configure the backend environment **.env** file for production, specifying your database path under **DATABASE_URL** (such as a local SQLite **drishyamitra.db** file or a production PostgreSQL instance). Host the Flask server using a WSGI server (such as **gunicorn** pre-installed in your requirements) and configure web servers to route incoming API queries to port **5000** while serving the static React build files, completing the deployment pipeline.

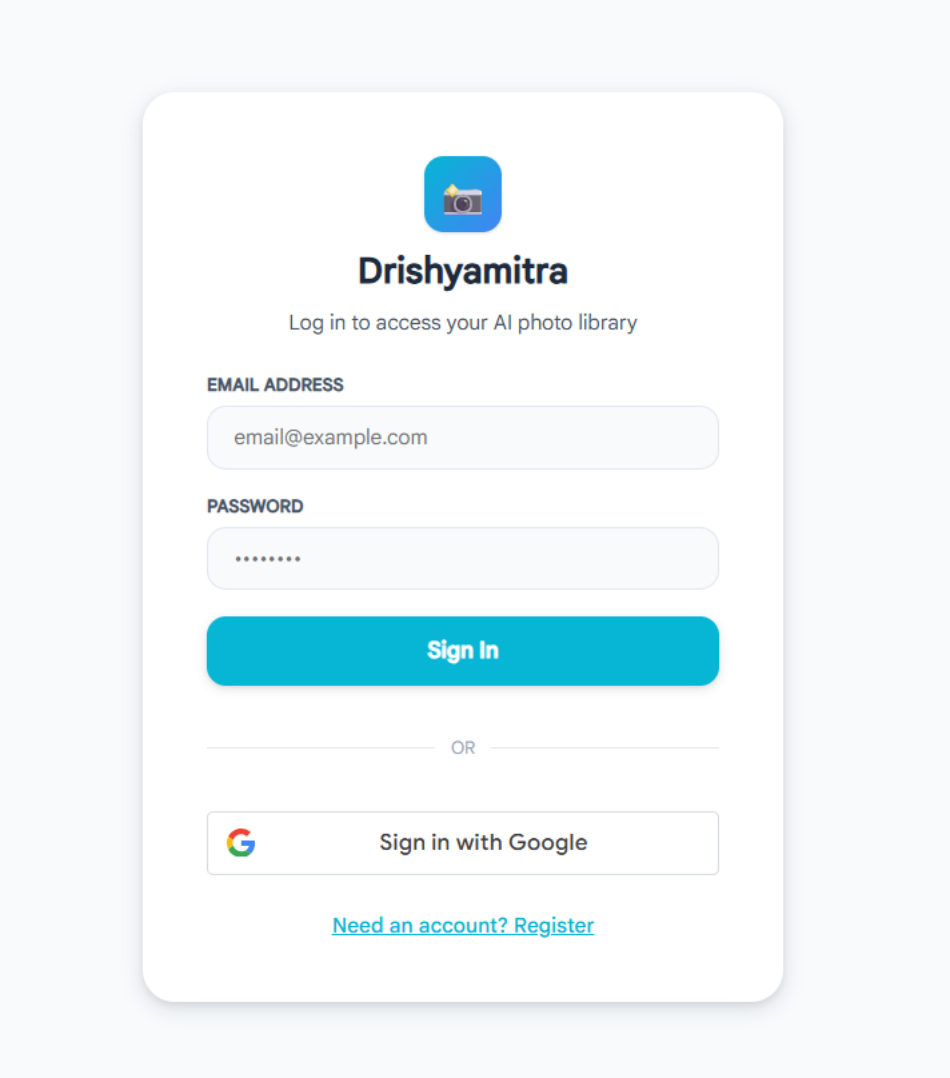
User Registration (Account Sign-Up) Interface

1. It provides form input fields for Username, Email Address, and Password, alongside integrated Google OAuth 2.0 verification to establish secure client sessions.
2. The overall card layout, typography, and Cyan register button are styled dynamically using the theme tokens configured in your project's theme.js file.

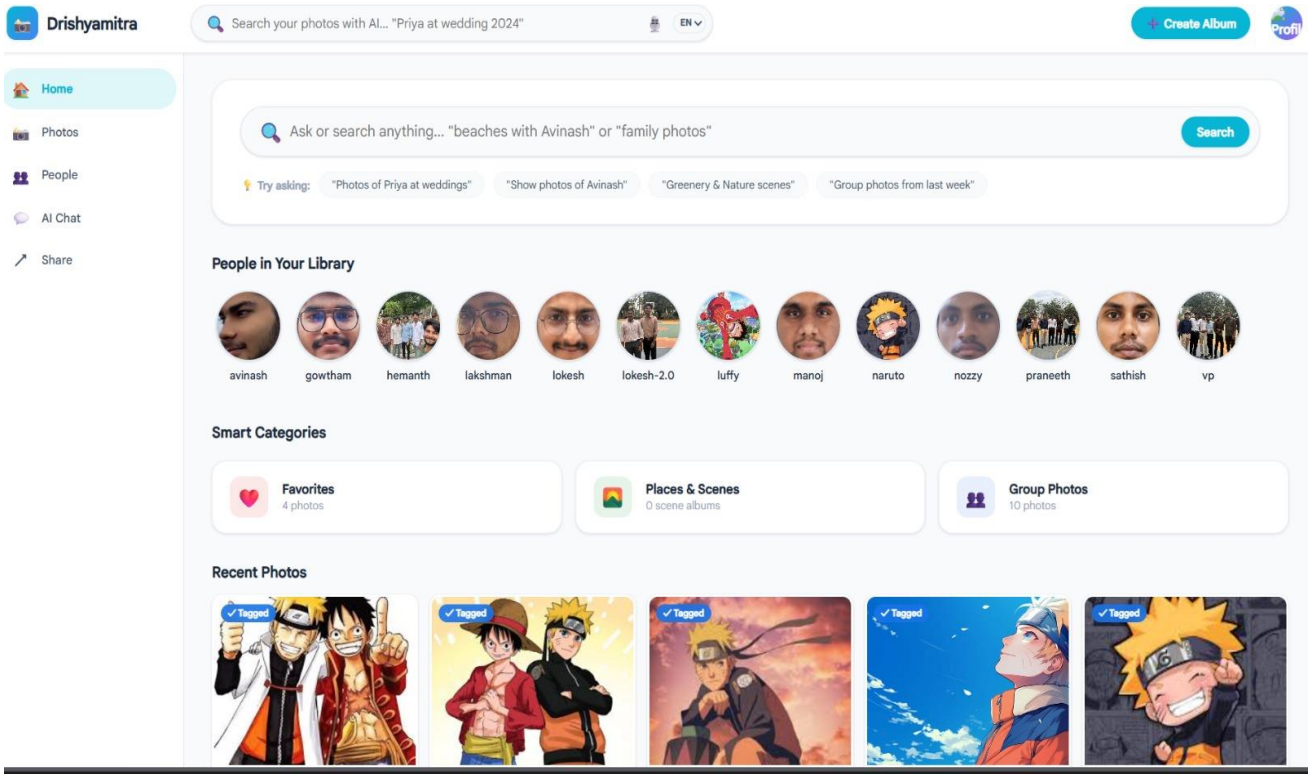


Sign-In Page:

- Secure Access: Uses JWT (JSON Web Token) authentication to protect private galleries and keep user data secure.
- Modern Design: Features a clean, responsive login interface styled with premium HSL color tokens and smooth animations.



Home Page:



Photos Page:

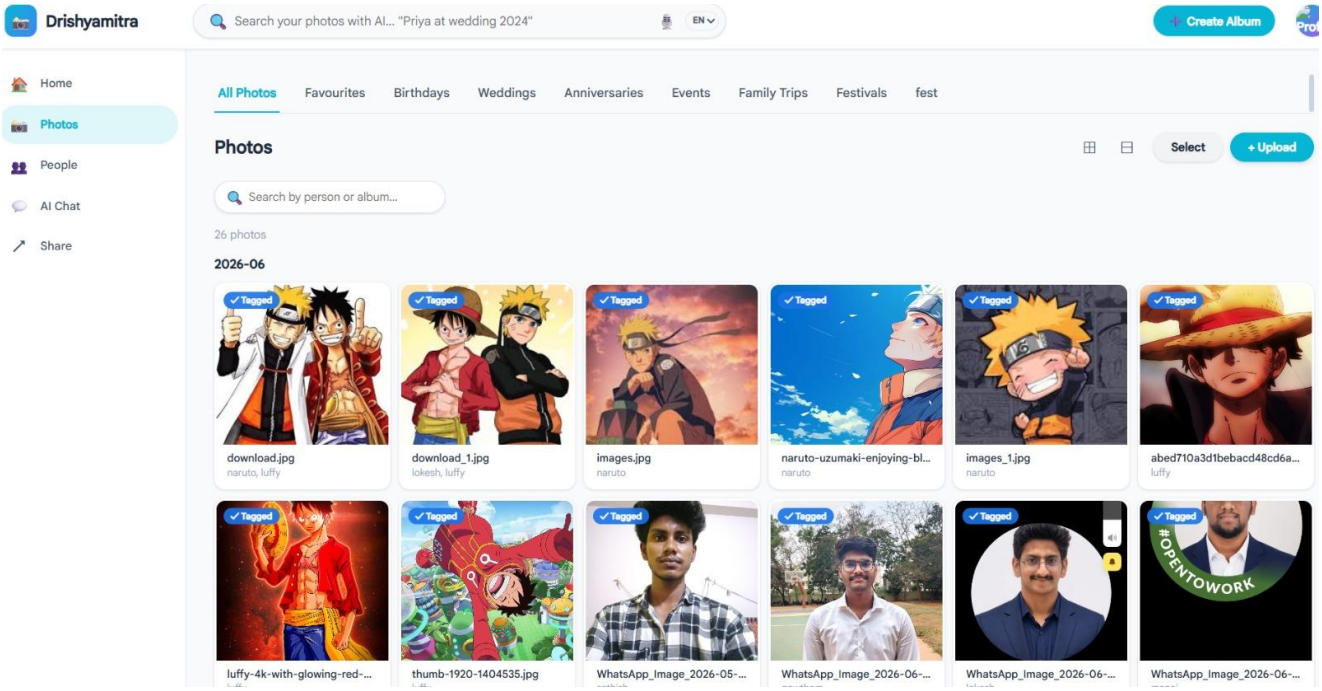
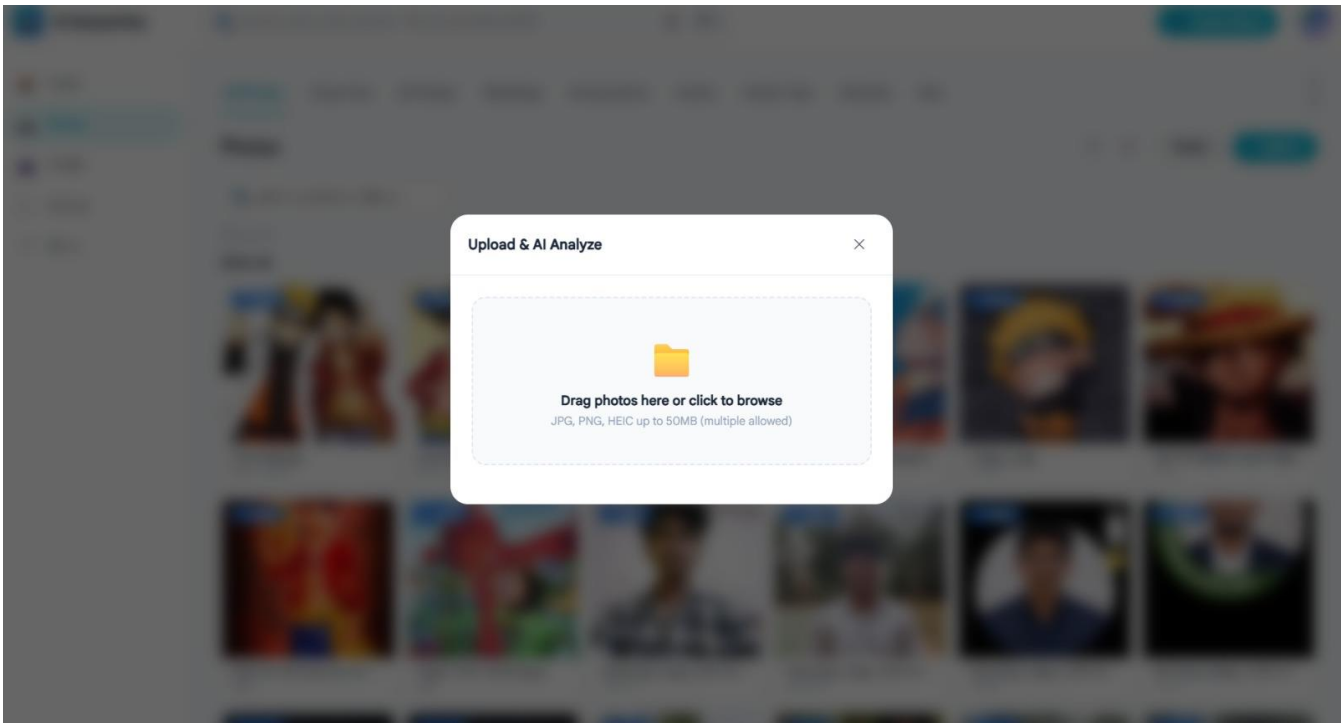
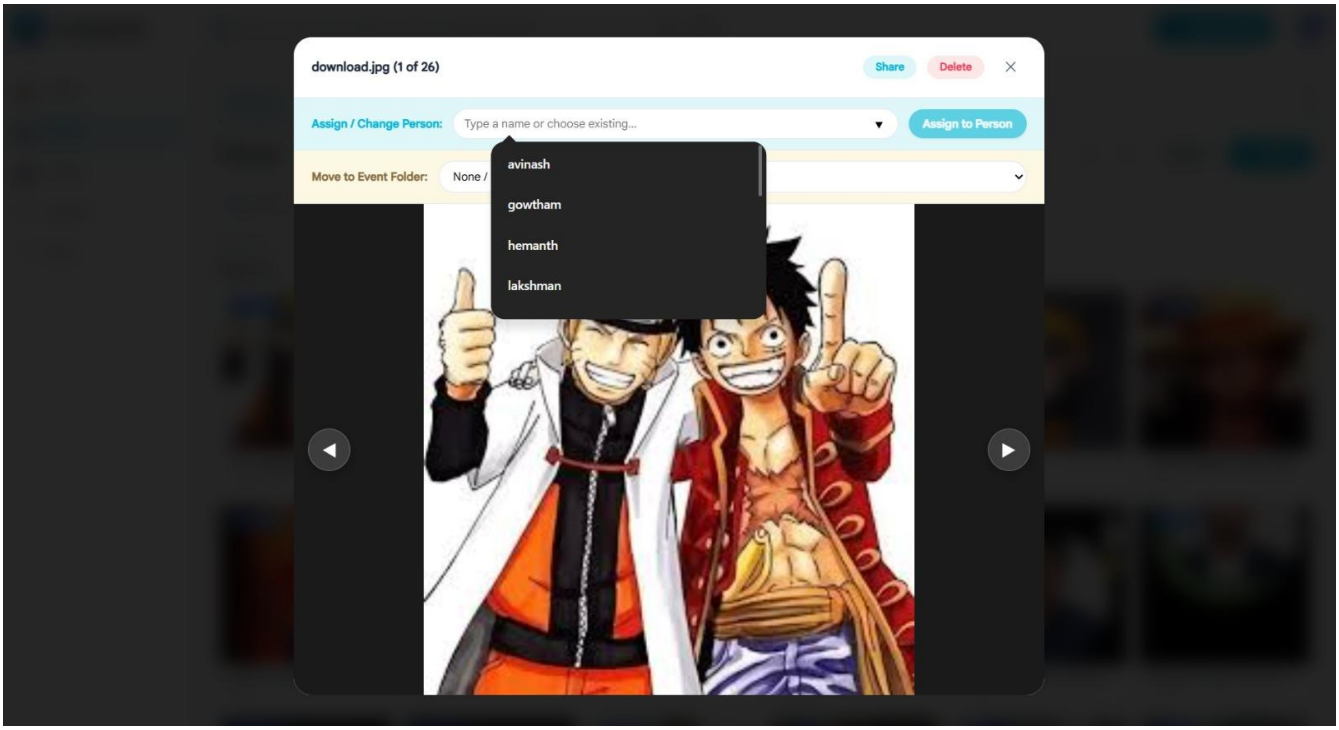
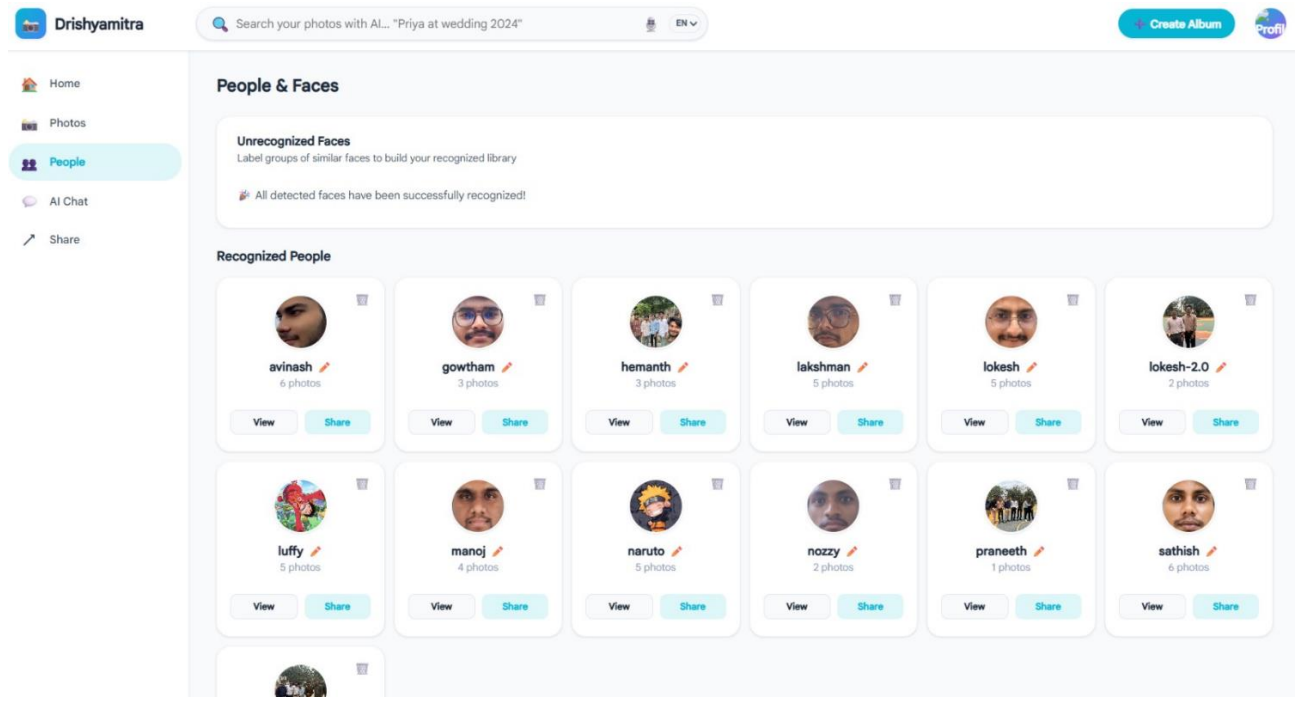


Photo Upload :

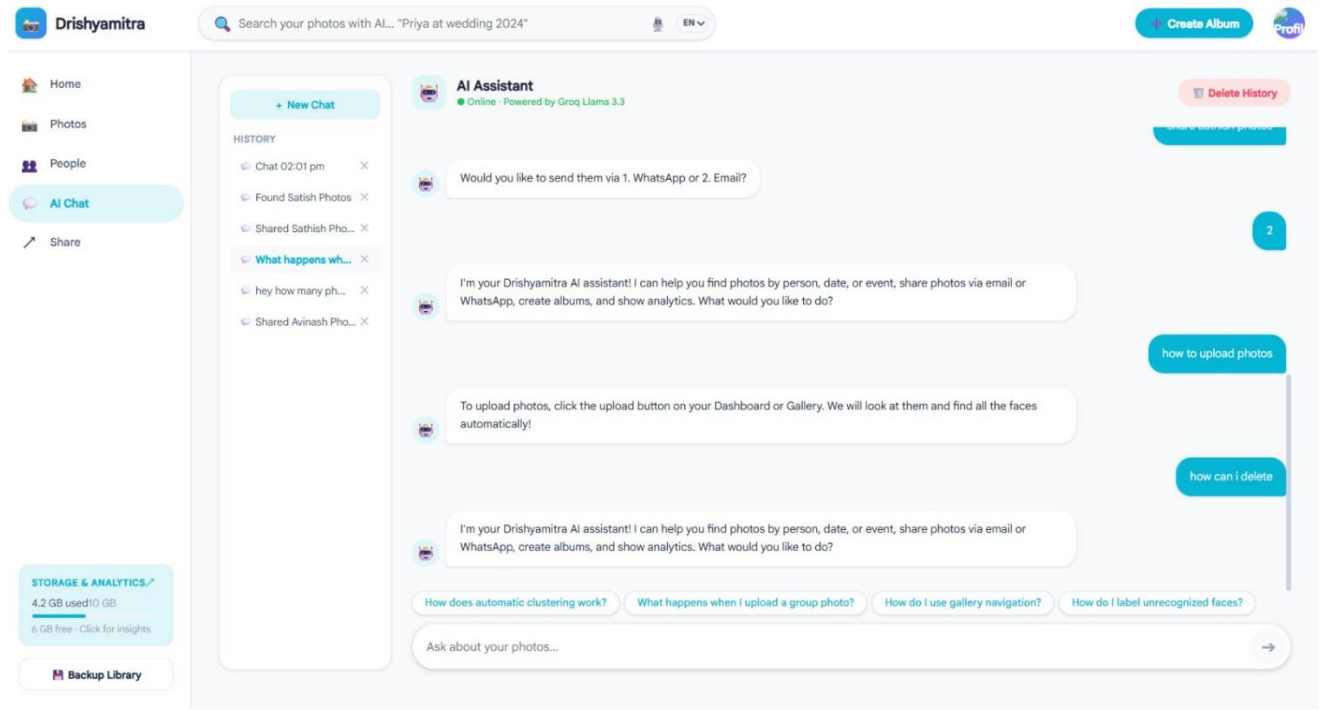




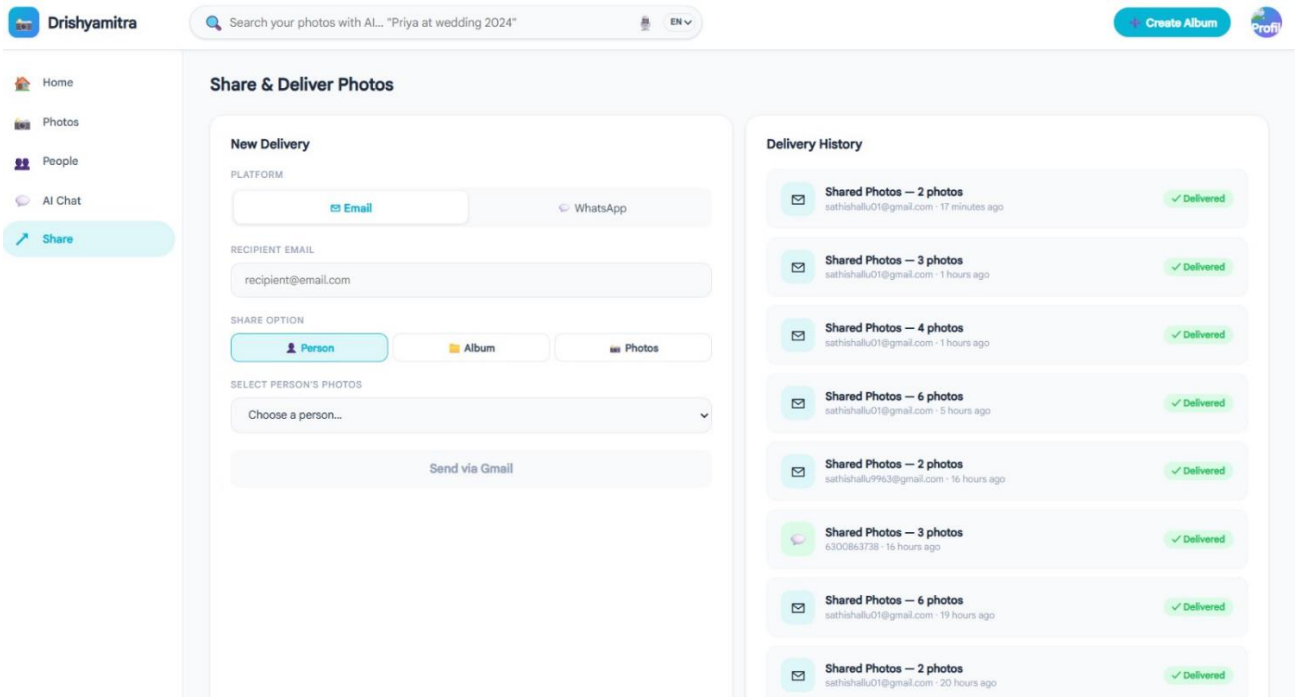
People Page:

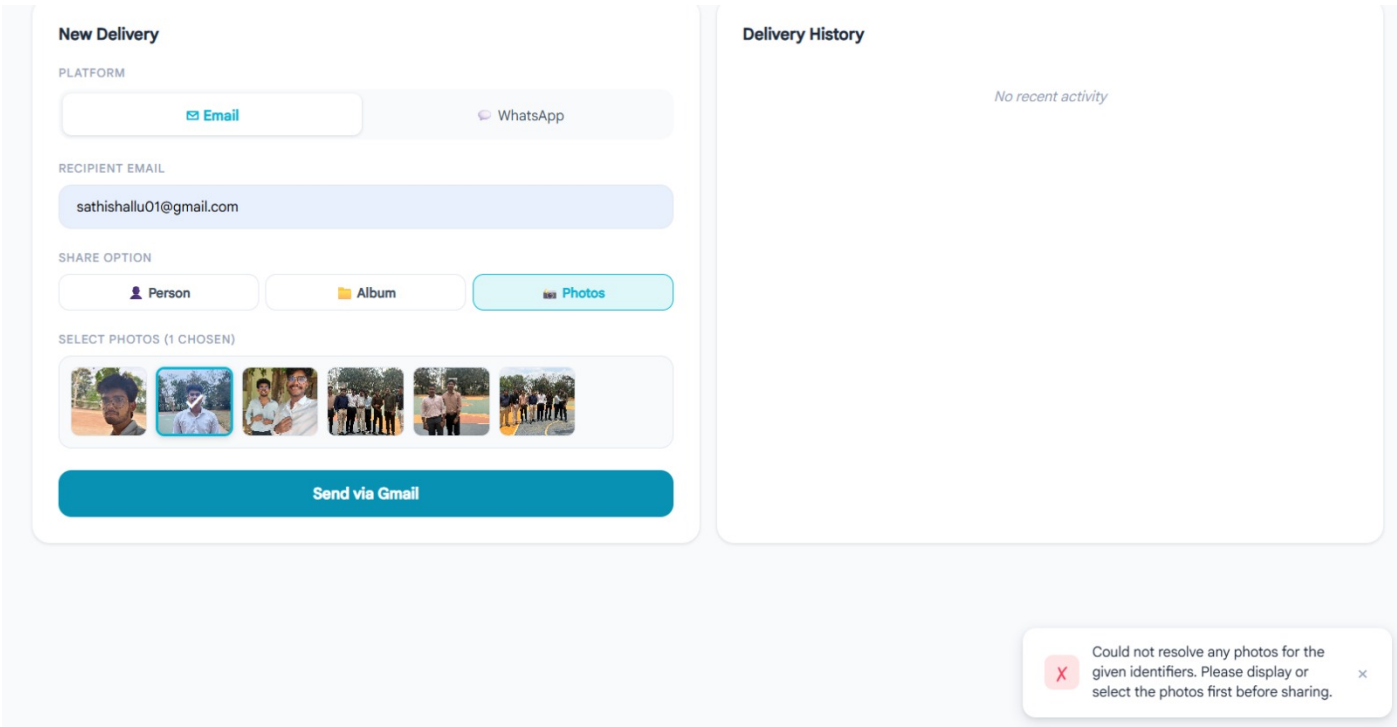


AI Chatbot Page :



Share Page :


Drishyamitra
Search your photos with AI... "Priya at wedding 2024"
EN
Create Album
Home
Photos
People
AI Chat
Share
Share & Deliver Photos
New Delivery
PLATFORM
Email
WhatsApp
RECIPIENT EMAIL
recipient@email.com
SHARE OPTION
Person
Album
Photos
SELECT PERSON'S PHOTOS
Choose a person...
Send via Gmail
Delivery History
Shared Photos — 2 photos
sathishallu01@gmail.com · 17 minutes ago
Delivered
Shared Photos — 3 photos
sathishallu01@gmail.com · 1 hours ago
Delivered
Shared Photos — 4 photos
sathishallu01@gmail.com · 1 hours ago
Delivered
Shared Photos — 6 photos
sathishallu01@gmail.com · 5 hours ago
Delivered
Shared Photos — 2 photos
sathishallu9963@gmail.com · 16 hours ago
Delivered
Shared Photos — 3 photos
6300863738 · 16 hours ago
Delivered
Shared Photos — 6 photos
sathishallu01@gmail.com · 19 hours ago
Delivered
Shared Photos — 2 photos
sathishallu01@gmail.com · 20 hours ago
Delivered


New Delivery
PLATFORM
Email
WhatsApp
RECIPIENT EMAIL
sathishallu01@gmail.com
SHARE OPTION
Person
Album
Photos
SELECT PHOTOS (1 CHOSEN)
Send via Gmail
Delivery History
No recent activity
Could not resolve any photos for the given identifiers. Please display or select the photos first before sharing.

Conclusion :

The development of **Drishyamitra** marks a significant step toward intelligent and automated photo management powered by modern AI technologies. By integrating advanced face recognition pipelines using **DeepFace (RetinaFace and Facenet512)** with a modular **Flask-React architecture**, the system delivers seamless automation—from detecting and cropping faces using OpenCV to grouping individuals into person-specific profiles. The inclusion of a natural-language-driven **AI Chat Assistant**, built on a **LangGraph state-machine workflow** and powered by **Groq Llama 3.3**, further enhances user interaction by allowing users to query, organize, and share photos conversationally rather than through complex menus.

Throughout its milestones, Drishyamitra achieved a robust backend capable of accurate face analysis, hybrid search combining **SQLite metadata** and **ChromaDB vector store** semantic embeddings, and multi-platform delivery via **secure Gmail SMTP** and **Twilio WhatsApp API** integrations. On the frontend, React.js styled dynamically with custom **Google Photos HSL design tokens** ensured a responsive, premium interface that keeps usability and performance at its core, while the integration of the browser **Web Speech API** provided multilingual voice search capabilities in English, Hindi, Spanish, and Telugu. Extensive testing across backend databases, frontend interfaces, and integrated workflows ensured the system's reliability, responsiveness, and user satisfaction.

Beyond its technical depth, Drishyamitra demonstrates how AI can simplify digital organization by merging automation, personalization, and accessibility. It effectively bridges the gap between advanced machine learning and everyday user needs, providing a practical tool that is both smart and user-friendly. The modular design also allows for future extensions, such as remote cloud synchronization, native mobile app interfaces, or deeper multimodal AI capabilities. In conclusion, Drishyamitra successfully embodies the vision of an intelligent photo companion—one that recognizes, organizes, and delivers with precision and ease.